Environment management for scientific computing

Risk assessment

- Installing software from unverified sources can introduce malware or compromise data integrity.
- Mixing system-wide and user-level installs can leave the system in a state that is hard to recover.

▷ Install package managers and packages only from reputable sources: official conda-forge and bioconda channels, the official miniforge installer, your operating system's vendor repositories.

▷ Avoid running unfamiliar shell installers piped from the internet without inspecting them first. Conda and mamba operate at the user level and should not require `sudo` (superuser privileges); if a command asks for it, treat that as a signal to stop and check.

Reviewed: May 10, 2026

This is a bench card. Full protocol available online.

›› *One-time install on a new machine*

(1.) Download the miniforge installer for the OS and CPU architecture from Miniforge, then run it.

```
bash Miniforge3-$(uname)-$(uname -m).sh
```

Critical: Do not run the installer with `sudo`. Conda installs under your user account and never needs administrator rights.

(2.) Open a new terminal. Verify the install and set the channel configuration.

```
mamba init
mamba info
mamba config --add channels bioconda
mamba config --add channels conda-forge
mamba config --set channel_priority strict
mamba install -n base conda-lock
```

›› *Per-project environment workflow*

(1.) Create one environment per project, named after the repository. Create it from an existing environment specification file `environment.yml`, or from scratch. Specify the Python (and R) version explicitly to avoid silent upgrades when conda-forge advances.

```
mamba env create -n my-project -f environment.yml
mamba activate my-project
```

If no environment file exists yet:

```
mamba create -n my-project python=3.12
mamba activate my-project
```

Critical: Keep the `base` environment empty except for mamba itself. Installing project-related packages into `base` is the most common path to a fragile installation.

(2.) Install all packages the project needs (Python libraries, R packages, and command-line tools), then export and lock the environment immediately.

```
mamba install numpy pandas scipy matplotlib samtools bwa multiqc
mamba env export --from-history > environment.yml
conda-lock lock -f environment.yml
```

> ***Critical:*** Install command-line bioinformatics tools (`samtools`, `bwa`, `bcftools`, `nextflow`, etc.) via mamba from bioconda, not through Homebrew or apt. System-installed tools are invisible to `environment.yml` and will not travel with the project.

(3.) When a new package is needed mid-project, install it, then re-export and re-lock as above.

Finding packages

(1.) Search for a package by name from the command line:

```
mamba search numpy
mamba search r-ggplot2
mamba search samtools
```

(2.) Alternatively, browse the channel web interfaces.

›› *Sharing and managing environment files*

(1.) When working with Git to track changes SOP 0101, commit both `environment.yml` and the lockfile to the project repository alongside the source code. When one changes, regenerate and commit the other in the same commit.

> ***Critical:*** Updating `environment.yml` without regenerating the lockfile leaves the two artifacts inconsistent. The next collaborator who installs from the lockfile will not see your changes to the environment file; the next who installs from `environment.yml` will get a different version set than you tested.

(2.) Pin top-level packages with major and minor versions in `environment.yml`; leave patches floating.

(3.) To recreate an environment exactly from a lockfile on a new machine, install from the lockfile rather than the environment file.

```
conda-lock install --name my-project conda-lock.yml
```

(4.) When working with an external repository, apply the same discipline. If forking to modify or extend: adopt the upstream `environment.yml` if one exists (create one if not), then immediately regenerate a lockfile in the fork.

```
mamba env create -f environment.yml
mamba activate my-project
```

Trial upgrade of a pinned dependency

(1.) Before changing anything, capture the current working state so the original environment can be rebuilt from scratch if both it and the trial are lost.

```
mamba env export -n my-project --from-history > environment.yml
conda-lock lock -f environment.yml
```

> ***Critical:*** Do not skip this step even if `environment.yml` and the lockfile look up to date. The lockfile is the actual safety net; everything that follows can be discarded as long as it remains valid.

(2.) Create a Git branch SOP 0101 so the YAML edit and any code changes the upgrade forces (deprecation fixes, syntax updates, dependency adjustments) stay isolated from the working `main` branch.

```
git checkout -b try-r45
```

(3.) Edit `environment.yml` on the branch. Bump the pin to try, and loosen any other pins likely to block the upgrade (typically transitive numerical or ML libraries with strict interpreter-ABI requirements).

```
# environment.yml - diff
```

```
- - r-base=4.4
+ - r-base=4.5
```

Regenerate the lockfile from the edited YAML. This is the feasibility test: if the solver cannot satisfy the new pin set, the failure surfaces here, before building any environment.

```
conda-lock lock -f environment.yml
```

(4.) Build the trial environment from the new lockfile in a single solve, under a new name.

```
conda-lock install --name my-project-r45 conda-lock.yml
mamba activate my-project-r45
```

Run the project's analyses against the trial env and confirm the results match the original. Commit any required code fixes to the `try-r45` branch along the way.

(5.) If the upgrade works, promote the trial: retire the original, rename the trial, and merge the branch.

```
mamba env remove -n my-project
mamba rename -n my-project-r45 my-project
git checkout main
git merge try-r45
```

The updated `environment.yml` and lockfile are already on the branch from the YAML-edit step, so the merge brings the spec across together with any code changes.

(6.) If the upgrade breaks, discard the trial environment and the branch. The original environment, `environment.yml`, lockfile, and `main` branch are untouched.

```
mamba env remove -n my-project-r45
git checkout main
git branch -D try-r45
```

Recovering a broken environment

(1.) When an env misbehaves (failed install, cache corruption, "nothing to do" when some packages are clearly missing, mysterious post-transaction states, accidental `pip install` etc.), delete and rebuild from the lockfile rather than debugging in place.

```
mamba deactivate
mamba env remove -n my-project
conda-lock install --name my-project conda-lock.yml
mamba activate my-project
```

If this succeeds, recovery is complete: the env is bit-identical to the committed state.

(2.) If no committed lockfile exists, or installing from it fails (channels evicted a pinned build, the host platform changed, etc.), regenerate a new lockfile from `environment.yml` first:

```
conda-lock lock -f environment.yml
conda-lock install --name my-project conda-lock.yml
mamba activate my-project
```

Commit the regenerated lockfile in the same operation. The project now has a working lockfile again.

> ***Critical:*** This path re-runs the solver against the current channel state. The resulting env may resolve to different transitive versions than the one that was working — especially after a Python or R minor version has advanced. If the regenerated lockfile fails to solve at all, the YAML's pins are jointly infeasible against current channels; loosen the offending pins (typically transitive numerical or ML libraries with strict interpreter-ABI requirements) deliberately, then re-lock.

Updating and pruning

(1.) Update mamba periodically. Update only in `base`, not in project environments.

```
mamba update -n base mamba
```

(2.) List existing environments and remove ones for projects no longer worked on.

```
mamba env list
mamba env remove -n old-project
```

(3.) Reclaim disk space from the package cache.

```
mamba clean --all
```

Resources (available online) Troubleshooting (available online) Notes (available online)

Lab Protocols — © B. C. Buchmuller. Licensed CC-BY-SA-4.0. Research use; provided as-is, without warranty. Current when printed. Visit https://benjbuch.github.io/check/?q=4166ca8 or scan the QR code to read the full protocol or to check for updates.